# iWithYou — 即时通信项目架构文档

> 最后更新：2026-03-27

## 一、项目概览

**iWithYou** 是一个全栈即时通信（IM）应用，采用 **Dart/Flutter** 技术栈，前后端语言统一。支持 Android、iOS、macOS、Web 多平台，提供文字/语音/图片/视频/文件等多种消息类型，以及好友系统、群聊、推送通知、AI 聊天等功能。

| 项目 | 技术 | 版本 |
|---|---|---|
| 后端 | Dart + Shelf | v1.4.3 |
| 前端 | Flutter + GetX | v2.0.87+220 |
| 数据库 | MySQL (utf8mb4) | — |
| 实时通信 | WebSocket (JSON) | — |
| 推送 | 华为/小米/OPPO/VIVO/荣耀/APNs | — |

---

## 二、整体架构

```
┌──────────────────────────────────────────────────────────────────────┐
│                              客户端 (Flutter)                          │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐                 │
│  │ 聊天模块 │ │ 好友模块 │ │ 发现模块 │ │ 个人中心 │                 │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘                 │
│       │            │            │            │                        │
│  ┌──────────────────────────────────────────────────┐                 │
│  │                GetX Controller 层                 │                 │
│  ├──────────────────────────────────────────────────┤                 │
│  │                Service / Repository 层            │                 │
│  ├──────────┬──────────┬──────────┬──────────────────┤                 │
│  │auth_client│ wxtchat  │chat_dtos │ wxtpush_client │                 │
│  └──────────┴──────────┴──────────┴──────────────────┘                 │
│       │ HTTP/JWT  │ WebSocket + HTTP        │ Native Push              │
└───────┼──────────┼──────────────────────────┼──────────────────────────┘
        │          │                          │
   ─────┼──────────┼──────────────────────────┼──── 网络 ────────────
        │          │                          │
┌───────┼──────────┼──────────────────────────┼──────────────────────────┐
│       ▼          ▼                          ▼                          │
│  ┌──────────────────────────────────────────────┐                      │
│  │            Shelf HTTP Server (:8080)          │                      │
│  │   ┌──────────────┐   ┌──────────────────┐    │                      │
│  │   │ Auth 中间件  │→  │ Router 路由分发  │    │                      │
│  │   │
```

```
/api/auth/*      /api/chat/*   /ws/chat
(认证路由)        (聊天API)     (WebSocket)

authcore    authsql    chatcore   wxtpushserver
(认证核心)  (SQL适配器) (IM核心)   (推送服务)

wxtmysql
(连接池 + 事务)

MySQL

服务端 (Dart)
```

---

## 三、后端架构

### 3.1 技术栈

- **语言**：Dart 3.2.1+
- **框架**：Shelf (轻量级 HTTP 框架) + Shelf Router
- **数据库**：MySQL (mysql1 驱动)
- **认证**：JWT (HS256)
- **序列化**：JSON + Protobuf (双协议)
- **WebSocket**：shelf*web*socket
- **部署**：Docker

### 3.2 入口与启动流程

入口文件 `backend/bin/server.dart` ，启动顺序：

1. **初始化日志** — 配置 INFO 级别日志
2. **创建数据库表** — `CREATE TABLE IF NOT EXISTS` （幂等）
3. **注册认证路由** — REST ( `/api/auth/*` ) + Protobuf ( `/api/proto/auth/*` )
4. **初始化 ChatCore** — 数据库配置、聊天仓库、推送服务、媒体处理
5. **挂载静态文件** — `/sango/` 、 `/sangoplay/` 、 `/mini/`
6. **注册 404 兜底** — 未命中路由返回 JSON 错误
7. **组装中间件管线** — 日志 → JWT 认证 → 应用路由

8. 启动 **HTTP** 服务 — 监听 `0.0.0.0:8080`
9. 注册信号处理 — SIGINT/SIGTERM 优雅停机

### 3.3 中间件管线

```
请求 → logRequests → authMiddleware → validationErrorMiddleware → Router
```

| 中间件 | 职责 |
|---|---|
| `logRequests` | 请求/响应日志记录 |
| `authMiddleware` | JWT 令牌验证，注入用户上下文（uid, deviceid） |
| `validationErrorMiddleware` | 捕获 JSON 校验错误，按客户端追踪错误模式 |

公开路径（无需认证）： `/health` 、 `/` 、 `/api/auth/*` 、 `/api/proto/auth/*`

### 3.4 API 路由

#### 公开接口

| 方法 | 路径 | 说明 |
|---|---|---|
| GET | `/` | 首页 (index.html) |
| GET | `/privacy` | 隐私政策 |
| GET | `/health` | 健康检查（状态、版本、数据库连通性） |
| GET | `/getvisit` | 获取访问计数 |
| POST | `/updatevisit` | 增加访问计数 |

#### 认证接口 (`/api/auth/*`)

| 方法 | 路径 | 说明 |
|---|---|---|
| POST | `/api/auth/login` | 邮箱/手机号登录 |
| POST | `/api/auth/register` | 注册（密码/手机/游客） |
| POST | `/api/auth/refresh` | 刷新令牌 |
| POST | `/api/auth/logout` | 登出 |
| POST | `/api/auth/oauth` | 第三方 OAuth 登录 |
| POST | `/api/proto/auth` | Protobuf 二进制认证（所有操作） |

### 用户接口

| 方法 | 路径 | 说明 |
|---|---|---|
| POST | `/api/users/me/profile` | 更新当前用户资料 |
| GET | `/api/users/me/profile` | 获取当前用户资料 |
| GET | `/api/users/<id>/profile` | 获取指定用户资料 |
| GET | `/api/users/<id>/details` | 获取用户详情（资料+位置） |
| POST | `/api/users/me/location` | 保存当前用户位置 |
| GET | `/api/users/<id>/location` | 获取用户位置 |

### 版本管理接口

| 方法 | 路径 | 说明 |
|---|---|---|
| POST | `/api/version` | 提交新版本信息 |
| GET | `/api/version/latest` | 获取最新版本与更新日志 |

### 聊天接口 *(由 ChatCore 插件动态注册)*

- `/api/chat/*` — 消息 CRUD、群组管理、好友操作、设备注册
- `/ws/chat` — WebSocket 实时通信

## 3.5 数据库模型

```
┌──────────────┐      ┌──────────────────┐      ┌──────────────┐
│  auth_users  │──────→│ auth_sessions    │      │auth_providers│
│  (用户表)    │      │  (会话表)        │      │ (OAuth映射)  │
├──────────────┤      ├──────────────────┤      ├──────────────┤
│ id (PK)      │      │ id (PK)          │      │ user_id (FK) │
│ email        │      │ user_id (FK)     │      │ provider     │
│ phone        │      │ token_hash       │      │ external_id  │
│ display_name │      │ ip, user_agent   │      └──────────────┘
│ avatar_url   │      │ expires_at       │
│ password_hash│      │ revoked_at       │
│ is_guest     │      └──────────────────┘
└──────────────┘
       │
       ├──→ user_profiles      (扩展资料: bio/birthday/gender/部门等)
       ├──→ Location           (GPS位置: 经纬度+地址)
       ├──→ chat_rooms         (聊天室)
       ├──→ room_members       (房间成员)
       ├──→ chat_messages      (聊天消息)
       ├──→ message_status     (消息状态: 已发/已达/已读)
```

```
        ├──→ offline_messages   (离线消息缓冲)
        ├──→ device_tokens      (设备推送令牌)
        ├──→ group_settings     (群组设置)
        └──→ user_groups        (用户已加入群组)

独立表:
  - visits     (网站访问统计)
  - versions   (版本信息与更新日志)
```

### 3.6 环境变量配置

| 变量 | 用途 |
|---|---|
| `MYSQL_HOST/PORT/DBNAME/USER/PASSWORD` | 数据库连接 |
| `JWTSECRET` | JWT 签名密钥 |
| `PORT` | 服务端口（默认 8080） |
| `WECHATAPPID/WECHATSECRET` | 微信 OAuth |
| `APNSKEYID/APNSTEAMID/APNSPRIVATEKEY` | Apple 推送 |
| `TCCLOUDID/TCCLOUDKEY` | 腾讯云凭证 |
| `umengAppKey/umengAppMasterSecret` | 友盟推送 |
| `VERSION_API_SECRET` | CI/脚本访问版本接口的密钥 |
| `LATESTVERSION/IOSURL/ANDROIDURL` | 应用版本与分发地址 |

---

## 四、后端插件体系

后端通过 5 个本地插件实现模块化，位于 `backend/plugins/` ：

### 4.1 authcore (v0.1.22) — 认证核心

职责：定义认证领域的接口、模型和业务逻辑。

- **AuthService** — 认证服务接口（注册、登录、刷新、登出）
- **UserRepository** — 用户数据访问接口
- **TokenService** — JWT 令牌生成与验证（HS256）
- **SessionStore** — 会话管理接口
- **DefaultAuthService** — 参考实现，支持单设备登录（新登录自动踢旧会话）
- **AuthRoutes / ProtoAuthRoutes** — REST (JSON) 和 Protobuf 双协议路由

特性：

- 邮箱 + 手机号 + 游客注册

- 微信/Google/Apple OAuth
- 邮箱/短信 OTP 验证
- JWT 自定义 Claims 扩展

### 4.2 authsql (v0.1.14) — MySQL 认证适配器

职责：`UserRepository` 和 `SessionStore` 的 MySQL 具体实现。

- 令牌以 SHA2(256) 哈希存储，绝不存明文
- `auth_users` 表自增 ID 从 10000 起
- 自动创建系统助手用户 (ID: 10000)
- 支持 UTF8MB4（emoji 兼容）
- 参数化查询防 SQL 注入

### 4.3 wxtmysql (v0.1.12) — MySQL 连接池

职责：高性能 MySQL 连接池管理与事务支持。

- **DatabaseService** 单例 — 提供 query/insert/update/delete/transaction
- **ConnectionPool** — 队列+锁实现，可配置最小/最大连接数
- **SemaphoreConnectionPool** — 备选信号量式连接池
- 自动回收空闲连接、按需创建新连接
- 周期性健康检查
- 连接池指标统计（总量/活跃/超时）
- SIGINT/SIGTERM 优雅关闭

### 4.4 chatcore (v1.0.39) — IM 核心

职责：完整的即时通信服务实现，是后端最核心的插件。

服务层：

| 服务 | 职责 |
|---|---|
| MessageService | 消息发送、接收、搜索、撤回 |
| RoomService | 房间 CRUD、成员管理 |
| UserService | 用户资料、在线状态 |
| FriendService | 好友列表、拉黑 |
| OfflineMessageService | 离线消息存储转发 |
| DeviceTokenService | 设备令牌注册 |
| ConnectionService | WebSocket 连接管理 |
| UnifiedPushService | 在线/离线推送调度 |
| DeepSeekService | AI 聊天机器人（系统房间） |
| MediaProcessingService | 图片/文件上传处理 |

**WebSocket 协议格式**：

```
{
  "type": "chat_message | user_online | user_offline | typing | error |
room_created",
  "messageData": { "id": "...", "roomId": "...", "content": "..." },
  "messageId": "...",
  "roomId": "...",
  "senderId": "...",
  "timestamp": "2026-03-27T10:00:00Z"
}
```

**房间类型**：私聊 (private)、群聊 (group)、频道 (channel)

**消息类型**：文字、图片、视频、语音、文件、位置、系统消息

### 4.5 wxtpushserver (v1.0.2) — 多厂商推送

**职责**：统一推送通知服务，支持 6 大平台。

| 厂商 | 服务类 |
|---|---|
| 华为 HMS | HuaweiPushService |
| 荣耀 | HonorPushService |
| 小米 | XiaomiPushService |
| OPPO | OppoPushService |
| VIVO | VivoPushService |
| Apple APNs | ApplePushService |

**PushService 单例提**供：`sendToToken()`、`sendToTokens()`、`sendToTopic()`、`sendToMultiVendorTokens()`

---

## 五、前端架构

### 5.1 技术栈

- **框架**：Flutter (Dart 3.2.1+)
- **状态管理**：GetX (^4.6.6)
- **路由**：GetX 命名路由 + Bindings
- **HTTP 客户端**：Dio (^5.9.0)
- **本地存储**：SQLite (sqflite) + SharedPreferences

### 5.2 应用架构模式

```
lib/
├── main.dart                    # 应用入口
```

```
├── configure.dart              # 全局配置（服务器地址、密钥等）
├── app/
│   ├── app_routes.dart         # 路由定义
│   ├── core/
│   │   ├── bindings/           # GetX 依赖注入
│   │   └── services/           # 核心服务层
│   │       ├── chat/           # 聊天网络配置
│   │       ├── push/           # 推送消息处理
│   │       ├── log/            # 日志服务
│   │       ├── theme_service     # 主题切换
│   │       ├── font_service      # 字体管理
│   │       └── location_service  # 地理位置
│   ├── modules/                # 功能模块（MVVM with GetX）
│   │   ├── home/               # 主页（含 4 个 Tab）
│   │   │   └── tabs/
│   │   │       ├── ChatTabView      # 聊天列表
│   │   │       ├── FriendsTabView   # 好友列表
│   │   │       ├── DiscoverTabView  # 发现页
│   │   │       └── MeTabView       # 个人中心
│   │   ├── login/              # 登录
│   │   ├── register/           # 注册
│   │   ├── settings/           # 设置
│   │   ├── user_detail/        # 用户详情
│   │   └── profile_edit/       # 资料编辑
│   ├── theme/                  # 主题定义（明/暗）
│   └── widgets/                # 通用组件
└── plugins/                    # 本地插件
```

模块内部遵循 **GetX MVC** 结构：

- **View** — UI 界面
- **Controller** (GetxController) — 业务逻辑 + 响应式状态
- **Binding** — 依赖注入配置

## 5.3 路由表

| 路径 | 模块 | 说明 |
|---|---|---|
| `/` | splash | 启动页 |
| `/home` | home | 主页（含 Tab 栏） |
| `/login` | login | 登录页 |
| `/register` | register | 注册页 |
| `/settings` | settings | 设置页 |
| `/about` | about | 关于页面 |
| `/profile-edit` | profile_edit | 编辑资料 |
| `/change-password` | — | 修改密码 |
| `/device-management` | — | 设备管理 |
| `/user-detail` | user_detail | 用户详情 |

## 5.4 网络通信

双协议架构：

| 协议 | 用途 | 配置 |
|---|---|---|
| HTTP REST (JSON) | 数据 CRUD、认证、文件上传 | Dio，`/api/*` |
| WebSocket (JSON) | 实时消息、在线状态、输入状 | `ws://host:port/ws/chat` |

网络参数：

- 请求超时：60s (debug) / 30s (prod)
- 连接超时：30s (debug) / 15s (prod)
- 最大并发请求：10
- 最大上传大小：100MB
- 图片压缩质量：85%

**WebSocket 重连策略**：

- 心跳间隔：90 秒
- 重连延迟：[2s, 5s, 15s, 20s, 30s]
- 最大重试次数：5

### 5.5 服务器配置

| 环境 | 地址 | 端口 |
| --- | --- | --- |
| 开发 | `43.139.137.75` | 8080 (本地) / 80 (远程) |
| 生产 | `www.ycwithyou.com` | 80 |

### 5.6 文件上传策略

- 主存储：阿里云 OSS（oss-cn-shenzhen, bucket: gudongmini）
- 备选：腾讯云 COS
- **CDN 域名**：https://pfoss.lccjapp.cn
- 后端上传：`/app/v2/upload/index`

### 5.7 国际化

- 支持语言：English ( `en` ) + 简体中文 ( `zh_CN` )
- 默认语言：English

### 5.8 主题系统

- 支持 明/暗 双主题
- 主题切换由 `ThemeService` 管理
- 动态颜色方案由 `ThemeColorService` 生成
- 自定义字体族支持
- 文本缩放锁定在 0.8x ~ 1.5x

---

## 六、前端插件体系

前端通过 6 个本地插件实现功能模块化，位于 `frontend/plugins/` ：

### 6.1 auth_client (v0.1.13) — 认证客户端

职责：双模式认证 (REST JSON + Protobuf) + OAuth 集成。

- **AuthController** (GetX) — 管理认证状态流转
  - 状态机：`unauthenticated → authenticating → authenticated | authError`
  - 方法：`guestLogin()` 、`userLogin()` 、`register()` 、`oauthLogin()` 、`logout()`
- **AuthApiService** — 网络请求，支持 JSON 和 Protobuf 双协议切换
- **UserInfo** — 统一用户模型，兼容多种字段名格式
- **Tokens** — JWT 令牌持有，自动刷新拦截器

- **OAuth 支持**：Google、Apple、微信

### 6.2 wxtchat / chatclient (v0.0.138) — IM 客户端核心

职责：实时消息引擎，WebSocket 通信、SQLite 本地持久化、聊天 UI 组件。

网络层：

- `ChatApi` (抽象接口) — 80+ 方法覆盖消息/群组/好友/设备
- `JsonChatApi` — REST JSON 实现
- `WebSocketManager` (单例) — WebSocket 全生命周期管理
  - 自动重连、心跳检测、离线消息队列
  - 连接状态：disconnected → connecting → connected → reconnecting → failed

数据层：

- `DatabaseManager` — SQLite 本地缓存
- `MessageRepository` 、 `GroupRepository` 、 `UserRepository` — 数据访问

控制器层 (GetX)：

- `MessageListController` — 消息列表 + 分页 + 已读状态
- `ChatRoomController` — 单个会话
- `GroupListController` / `GroupDetailController` — 群组管理
- `FriendListController` / `FriendRequestController` — 好友管理

**UI 组件**：

- `ChatRoomView` — 消息列表界面
- `ChatInputBar` — 消息输入栏
- `SmartImageWidget` — 优化图片加载/缓存
- `GroupListView` 、 `FriendListPage` — 列表界面

上传策略：

- `UploadStrategyManager` — 支持腾讯 COS / 阿里 OSS 切换

### 6.3 chat_dtos (v1.0.0) — 共享数据传输对象

职责：前后端共享的数据协议定义。

- **ChatResult\<T>** — 统一 API 响应包装
- **WebSocketMessage** — WebSocket 协议封装（类型 + 事件 + 数据负载）
- **ChatMessageBase** — 消息抽象（支持回复、@提及、表情反应）
- 消息类型枚举：text, image, video, audio, file, emoji, link
- 事件类型枚举：userConnected, userDisconnected, messageRecall, ...
- **9 组请求 DTO**：auth / message / device / group / friend / push / member
- **5 组响应 DTO**：user / message / group / friend / stats

### 6.4 wxtpush_client (v1.1.1) — 多厂商推送客户端

职责：统一封装 6 大推送厂商的原生 SDK。

- **WxtpushClient** (单例) — initialize / requestPermission

- **PushConfig** — 多厂商配置（华为/小米/OPPO/VIVO/荣耀/Apple）
- **PushMessageHandler** (抽象) — 回调接口
  - `onMessageReceived`、`onMessageClicked`、`onTokenUpdated`、`onError`
- **DeviceUtils** — 自动检测当前设备支持的推送厂商
- 实现方式：Kotlin/Swift 原生代码 + Method Channel

### 6.5 jpushclient (v0.0.1) — 极光推送

职责：极光推送 Flutter SDK 封装。

- **PushService** (GetX Service) — 自动初始化、获取 registrationId
- **PushMessage** — 推送消息模型 (title, body, extras)
- 事件监听：通知接收、通知点击、透传消息

### 6.6 video_player_android_local (v2.9.4) — 自定义视频播放器

职责：Android 端视频播放实现，修复 HEVC 编解码兼容性。

- 基于 `video_player` 平台接口实现
- Pigeon 生成类型安全的 Method Channel
- 支持 TextureView / SurfaceView 渲染切换

---

## 七、核心数据流

### 7.1 用户登录流程

```
客户端                    服务端
  │                                        │
  ├── POST /api/auth/login ──────────────→│
  │   {email, password}                    │
  │                                        ├── AuthRoutes
  │                                        ├── AuthService.loginPassword()
  │                                        ├── MySqlUserRepository.getByEmail()
  │                                        ├── PasswordHasher.verify()
  │                                        ├── 踢掉旧会话 (单设备策略)
  │                                        ├── MySqlSessionStore.create()
  │                                        ├── TokenService.issueAccessToken()
  │                                        │
  │←── 200 {user, tokens} ──────────────┤
  │                                        │
  ├── 保存 tokens 到 SharedPreferences │
  ├── 建立 WebSocket 连接 ──────────────→│
  │   Authorization: Bearer <token>        │
  │                                        │
  │←── WebSocket Connected ─────────────┤
  │                                        │
  ├── 注册设备推送令牌 ──────────────────→│
  │   POST /api/chat/devices               │
```

```
  │                                    │
```

### 7.2 消息收发流程

```
发送者          服务端          接收者
  │                            │                          │
  ├── WebSocket send() ──→│                          │
  │   {type: chat_message}   │                          │
  │                            ├── MessageService         │
  │                            ├── 持久化到 chat_messages │
  │                            ├── ConnectionService      │
  │                            │   检查接收者是否在线   │
  │                            │                          │
  │                      [在线] ├── WebSocket broadcast →│
  │                            │                          │
  │                      [离线] ├── OfflineMessageService│
  │                            ├── UnifiedPushService     │
  │                            ├── wxtpushserver 推送 ──→│ (系统推送通知)
  │                            │                          │
  │←── 消息送达确认 ──────────┤                          │
  │                            │                          │
  │                            │←── 消息已读回执 ──────────┤
  │←── 已读状态更新 ──────────┤                          │
```

### 7.3 离线消息同步

```
客户端              服务端
  │                                    │
  ├── WebSocket 重新连接 ──────────────→│
  │                                    │
  │                                    ├── OfflineMessageService
  │                                    ├── 查询离线消息队列
  │                                    │
  │←── 批量推送离线消息 ────────────────┤
  │                                    │
  ├── 写入本地 SQLite ──────────────── │
  ├── 更新 UI ──────────────────────── │
  ├── 发送已送达确认 ──────────────────→│
```

---

## 八、部署架构

```
┌──────────────────────────────────────────┐
│              Docker Container            │
│   ┌──────────────────────────────────┐   │
│   │     Dart Shelf Server (:8080)    │   │
│   │     ┌──────────────────────────┐ │   │
│   │     │ HTTP + WebSocket 服务    │ │   │
```

```
│   │     │ 静态文件托管 (sango/mini)    │   │ │
│   │     └──────────────────────────┘   │ │
│   └──────────────────────────────────┘ │
│                    │                    │
│                    ▼                    │
│           MySQL Database                │
└──────────────────────────────────────┘
         │              │
         ▼              ▼
   ┌──────────┐   ┌──────────────┐
   │ 阿里 OSS  │   │ 推送厂商网关 │
   │ (文件存储) │   │ HMS/APNs/.. │
   └──────────┘   └──────────────┘
```

- **Dockerfile** 位于 `backend/Dockerfile`
- 启动脚本：`backend/run_New.sh`
- 发布脚本：`backend/release.command`

---

## 九、项目目录总览

```
iwithyou/
├── backend/                      # 后端服务
│   ├── bin/server.dart           # 服务入口
│   ├── lib/
│   │   ├── configure.dart        # 环境变量配置
│   │   ├── logger.dart           # 日志配置
│   │   ├── middleware/           # 中间件（认证、校验）
│   │   ├── model/                # 数据模型
│   │   ├── routes/               # API 路由
│   │   └── deprecated_folder/    # 已弃用代码
│   ├── plugins/
│   │   ├── authcore/             # 认证核心框架
│   │   ├── authsql/              # MySQL 认证适配器
│   │   ├── wxtmysql/             # MySQL 连接池
│   │   ├── chatcore/             # IM 核心引擎
│   │   └── wxtpushserver/        # 多厂商推送服务
│   ├── sango/                    # Web 管理界面
│   ├── sangoplay/                # Flutter Web 应用
│   ├── mini/                     # 轻量级前端/小程序
│   └── Dockerfile                # Docker 部署配置
│
├── frontend/                     # 前端应用
│   ├── lib/
│   │   ├── main.dart             # 应用入口
│   │   ├── configure.dart        # 全局配置
│   │   └── app/
│   │       ├── app_routes.dart   # 路由表
```

```
│   │       ├── core/            # 核心服务层
│   │       ├── modules/         # 功能模块（MVC）
│   │       ├── theme/           # 主题定义
│   │       └── widgets/         # 通用组件
│   ├── plugins/
│   │   ├── auth_client/         # 认证客户端
│   │   ├── chatclient/          # IM 客户端核心（wxtchat）
│   │   ├── chat_dtos/           # 共享数据协议
│   │   ├── wxtpush_client/      # 多厂商推送客户端
│   │   ├── jpushclient/         # 极光推送
│   │   └── video_player_android_local/  # 自定义视频播放器
│   ├── android/                 # Android 平台配置
│   ├── ios/                     # iOS 平台配置
│   ├── macos/                   # macOS 平台配置
│   ├── web/                     # Web 平台配置
│   └── assets/                  # 资源文件
│
└── docs/                        # 项目文档
```

## 十、关键技术决策

| 决策 | 选择 | 理由 |
|---|---|---|
| 全栈语言统一 | Dart | 前后端共享 DTO，降低协议不一致风险 |
| 状态管理 | GetX | 轻量级，内置路由/DI/国际化 |
| 双协议支持 | JSON + Protobuf | JSON 调试方便，Protobuf 高效传输 |
| 单设备登录 | 新登录踢旧会话 | IM 场景安全需求 |
| 推送适配 | 6 大厂商原生 SDK | 国内 Android 碎片化，需各厂商通道保证送达率 |
| 本地缓存 | SQLite | 离线消息可用，减少网络请求 |
| 令牌存储 | SHA2 哈希 | 数据库泄露不暴露原始令牌 |
| 连接池 | 自研队列+锁 / 信号量 | 可切换策略，针对 Dart async 特性优化 |
| 文件存储 | 阿里云 OSS (主) + 腾讯 COS (备) | 多云容灾 |